
Área Científica	Programação de Sistemas de Informação
Unidade Curricular	Introdução às Linguagens e Tecnologias Web
Ano Letivo / Semestre	2025/2026 – 2º Semestre
Docente Responsável	Prof. ^a Catarina Silva
Documento	Guião Prático Laboratorial 2
Tema	<i>Arquitetura & Client-Side: Arquitetura Web & Programação Client-Side</i>

Objetivos do Guião Prático

Este guião prático foi desenhado para consolidar a transição entre a construção de interfaces estáticas (HTML/CSS) e a programação dinâmica do lado do cliente (Client-Side).

- Selecionar, ler e alterar elementos HTML dinamicamente.
- Reagir a ações do utilizador (cliques, introdução de texto, submissão de dados).
- Utilizar variáveis para guardar estados e implementar estruturas de controlo de fluxo (condicionais e ciclos) na resolução de problemas práticos.

Estrutura Base do Projeto

Para garantir a correta separação de responsabilidades (HTML para a estrutura e JS para o comportamento), é fundamental organizar o código em ficheiros distintos.

Esta estratégia de posicionamento garante que o *browser* carrega e constrói primeiro toda a Árvore DOM; deste modo, quando o interpretador de JavaScript for executado, os elementos HTML já existem na memória.

Ficheiro HTML (index.html)

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">
  <title>Título do Exercício</title>
</head>
<body>
  <h1>A Minha Aplicação</h1>
  <input type="text" id="meuInput" placeholder="Insira algo...">
  <button id="meuBotao">Executar</button>

  <div id="resultado"></div>

  <script src="script.js"></script>
</body>
</html>
```

Ficheiro JavaScript (script.js)

```
// PASSO 1: Encontrar/Selecionar os Elementos HTML no DOM
let inputElement = document.getElementById("meuInput");
let btnElement = document.getElementById("meuBotao");
let divResultado = document.getElementById("resultado");

// PASSO 2: Escutar o Utilizador (Event Listeners)
btnElement.addEventListener("click", function() {

  // PASSO 3: Lógica e Manipulação
  // a) Ler o valor que o utilizador escreveu no input
  let valorInserido = inputElement.value;

  // b) Executar a lógica de programação (cálculos, validações, etc.)

  // c) Alterar o DOM (injetar o resultado no HTML)
  divResultado.innerHTML = textoFinal;

});
```

Exercício 1: Calculadora de Soma Simples

Em JavaScript, todos os valores retirados de campos de texto (<input>) são, por defeito, do tipo *String* (texto). Se tentarmos somar "1" + "2", o resultado será "12" (concatenação) e não 3. É necessário aplicar **coerção de tipos explícita**, utilizando funções

como `parseFloat()` ou `Number()`, para converter o texto em valores numéricos antes da operação matemática.

Enunciado: Crie uma página web que permita ao utilizador inserir dois números. A página deve ter um botão que, ao ser clicado, calcule a soma algébrica desses dois valores e apresente o resultado final no ecrã.

1. No HTML, crie dois `<input type="number">` (atribua-lhes IDs distintos, ex: `num1` e `num2`), um `<button>` e uma `<div>` ou `<span>` para o resultado.
2. No JavaScript, utilize `document.getElementById()` para referenciar os elementos HTML.
3. Adicione um *Event Listener* de clique (`"click"`) ao botão.
4. Dentro da função do evento, extraia os valores dos inputs utilizando a propriedade `.value`.
5. Converta os valores lidos para números e some-os.
6. Altere o `innerHTML` do elemento de resultado para exibir a soma final.

Exercício 2

A interatividade imediata exige que o JavaScript oiça eventos de forma contínua, reagindo a ações do utilizador. Enquanto o evento `"click"` reage a uma ação pontual num botão, o evento `"input"` é disparado sempre que o valor de um campo de texto é alterado. A propriedade `.length` de uma *String* devolve automaticamente o número de caracteres que compõem esse texto.

Enunciado: Desenvolva uma interface com uma área de texto grande (`textarea`). À medida que o utilizador escreve nessa área, a página deve calcular e exibir automaticamente, em tempo real, o número total de caracteres já digitados.

1. No HTML, crie uma `<textarea>` (atribua-lhe um ID) e um elemento de texto (ex: `<p id="contador">`) para apresentar a contagem.
2. No JavaScript, utilize `document.getElementById()` para selecionar a `textarea` e o elemento de contagem.
3. Adicione um *Event Listener* do tipo `"input"` à `textarea`.
4. Dentro da função do evento, obtenha o comprimento do texto atual acedendo a `textarea.value.length`.
5. Atualize o conteúdo interno (`innerHTML`) do elemento de texto com o novo valor calculado.

Exercício 3

Para além dos campos de texto normais, o JavaScript consegue ler valores de menus pendentes (`<select>`). Uma vez que o JavaScript trata operações numéricas com o tipo primitivo *Number* (incluindo decimais/floats), resultados monetários podem gerar dízimas longas. Utiliza-se o método `.toFixed(2)` para formatar o número resultante, garantindo que apresenta sempre duas casas decimais.

Enunciado: Construa um simulador onde o utilizador insere o valor total de uma refeição e escolhe a percentagem da gorjeta (ex: 5%, 10%, 15%, 20%) através de um menu pendente. Ao submeter a ação, a página deve apresentar o valor exato da gorjeta e o valor total final a pagar.

1. No HTML, crie um `<input type="number">` para a refeição, um `<select>` com opções de percentagem (ex: `<option value="0.10">10%</option>`) e um `<button>`.
2. Adicione elementos vazios no HTML para imprimir o valor da gorjeta e o total final.
3. No JS, adicione um evento de clique ao botão e garanta a conversão do valor da refeição para número.
4. Calcule a gorjeta multiplicando o valor da refeição pela percentagem selecionada.
5. Some a gorjeta ao valor da refeição para obter o total, e utilize `.toFixed(2)` ao atualizar o `innerHTML` dos resultados no ecrã.

Exercício 4

O controlo de fluxos altera a leitura linear do código. A utilização de condicionais (`if`, `else if`, `else`) permite saltar blocos de código e tomar decisões consoante a avaliação verdadeira ou falsa de uma condição. O JavaScript permite ainda alterar não só o conteúdo, mas também o CSS em tempo real, acedendo diretamente à propriedade `style` dos elementos do DOM.

Enunciado: Crie um sistema que peça ao aluno a sua nota final (um valor numérico de 0 a 20). Ao avaliar a nota, a interface deve exibir se o aluno está "Reprovado"(< 10), com nota "Suficiente"(10 a 13), "Bom"(14 a 17) ou "Excelente"(≥ 18). O texto deve assumir cores diferentes consoante a categoria.

1. Crie o `<input>` numérico, um botão para avaliar e uma `<div>` para a mensagem de saída.
2. No JS, após ler e converter a nota, inicie uma estrutura condicional combinando `if` e `else if`.
3. Utilize operadores de comparação (como `>=` e `<`) para delimitar os intervalos de notas.
4. Em cada bloco condicional, atualize o `innerHTML` do resultado com o texto adequado e modifique a cor aplicando `elementoResultado.style.color` (ex: "red", "orange", "blue", "green").

Exercício 5

Os operadores lógicos são essenciais para validar múltiplas premissas. O operador **AND** (`&&`) garante que o bloco de código só avança se *ambas* as condições testadas forem simultaneamente verdadeiras. Para melhorar a usabilidade em caso de erro, podemos forçar a limpeza de um campo de texto pelo JavaScript, atribuindo uma *string* vazia (`()`) à sua propriedade `value`.

Enunciado: Desenvolva um formulário de login simples. Assuma no código um par de credenciais válidas à sua escolha (ex: admin / 1234). Ao submeter, o sistema deve apresentar "Acesso Concedido" (a verde) se estiver correto, ou "Credenciais Inválidas" (a vermelho) em caso de erro. Em caso de falha, o campo da password deve ser automaticamente limpo.

1. No HTML, crie os `<input>` para utilizador e palavra-passe (`type="password"`), e o botão de submissão.
2. No JS, selecione os elementos e leia os seus valores no evento de clique do botão.
3. Construa um `if` utilizando o operador de igualdade estrita (`===`) aliado ao operador lógico `&&` para validar as duas credenciais na mesma linha.
4. No bloco de sucesso (`if`), apresente a mensagem de boas-vindas a verde.
5. No bloco de falha (`else`), apresente a mensagem de erro a vermelho e limpe a password executando `inputPassword.value = ''`.

Exercício 6

A programação eficiente baseia-se no princípio DRY (*Don't Repeat Yourself*). Quando necessitamos de repetir um bloco de código um número exato e conhecido de vezes, utilizamos o ciclo `for`. Podemos usar a iteração para gerar texto estruturado em HTML de forma dinâmica, concatenando as *Strings* volta após volta, antes de as injetar no DOM.

Enunciado: Crie uma aplicação onde o utilizador possa inserir um número inteiro. Ao gerar, a página deve calcular e apresentar a tabuada completa desse número (multiplicações do 1 ao 10) num formato de lista.

1. Crie no HTML o campo de entrada numérica, um botão e uma `<ul id="listaTabuada">` vazia.
2. No evento de clique em JS, crie uma variável textual inicializada vazia (`let htmlResultado = ''`).
3. Construa um ciclo `for` que declare um contador `i = 1` e itere até `i <= 10`.
4. Dentro do ciclo, multiplique o número inserido por `i` e adicione a nova linha à *string* acumuladora utilizando o operador `+=` (ex: `htmlResultado += <li>"+ ... + </li>`).
5. Após terminar o ciclo, atribua o `htmlResultado` ao `innerHTML` da lista.

Exercício 7

O JavaScript atua como manipulador capaz de criar coisas novas depois da página já estar carregada. Como a página HTML é lida sob a forma de uma Árvore DOM, podemos instanciar novos nós em tempo de execução e anexá-los a elementos parentais existentes, expandindo ativamente a estrutura visual da interface sem recarregamentos.

Enunciado: Desenvolva uma interface com um campo de texto e um botão. Quando o utilizador escreve e clica no botão, a entrada deve ser adicionada como um novo item a uma lista visual na página. O campo deve limpar-se automaticamente após a inserção.

1. Estructure um `<input type="text">`, um botão e uma `<ul id="tarefas">` no HTML.
2. No JS, dentro do *Event Listener* do botão, capture o texto inserido.
3. Utilize `document.createElement("li")` para gerar virtualmente um novo elemento de lista na memória.
4. Atribua o texto capturado à propriedade `innerText` deste novo elemento.
5. Selecione a `<ul>` com o seu ID e utilize o método `.appendChild(novoItem)` para fixar o elemento na árvore DOM.
6. Limpe o campo de entrada atribuindo ao seu `value`.

Exercício 8

Para imitar o comportamento de uma Inteligência Artificial básica ou adicionar incerteza, utilizamos a matemática do JavaScript. A conjugação de `Math.random()` (que devolve um valor decimal entre 0 e 1) com `Math.floor()` permite gerar números inteiros aleatórios que, aliados a uma estrutura condicional robusta de `if / else if / else`, definem os resultados das regras do jogo.

Enunciado: Construa um minijogo interativo com botões para Pedra, Papel e Tesoura. Ao clicar, gere aleatoriamente a jogada do "computador", aplique as regras do jogo para determinar o vencedor e exiba no ecrã o resultado final (Vitória, Derrota ou Empate).

1. Crie três botões distintos no HTML (Pedra, Papel, Tesoura) e uma `<div>` para relatar os resultados da ronda.
2. No JS, adicione *Event Listeners* que identifiquem qual a escolha do jogador.
3. Gere a jogada do computador criando um número aleatório entre 0 e 2 (onde 0=Pedra, 1=Papel, 2=Tesoura).
4. Utilize estruturas de comparação (operador de igualdade `===`) num bloco condicional para verificar primeiro se houve Empate.
5. Utilize `else if` para definir explicitamente as 3 condições de Vitória do utilizador.

6. Utilize um **else** final para assumir todas as restantes combinações como Derrota e atualize a mensagem no ecrã.

Exercício 9

Variáveis funcionam como caixas de memória. Para preservar o estado (os dados) de uma aplicação *Client-Side* de forma transversal, devemos declarar as variáveis de controlo no escopo global (fora das funções) utilizando **let**. Desta forma, o seu valor persiste e pode ser acumulado ou subtraído ao longo de múltiplas interações.

Enunciado: Crie uma interface que liste três produtos com preços fixos (ex: Rato 10€, Teclado 25€). Permita ao utilizador indicar quantidades a comprar. Ao adicionar, a página deve calcular o custo dessa parcela, somá-lo ao total anterior e exibir o valor atualizado do "carrinho".

1. No ficheiro JS, declare uma variável global (ex: `let totalCarrinho = 0;`) no topo do documento.
2. No HTML, para cada produto, crie um `<input type="number">` (para a quantidade) e um botão "Adicionar".
3. No JS, crie funções de clique para os botões. Obtenha a quantidade inserida e converta-a para número.
4. Multiplique a quantidade pelo preço fixo definido para esse produto.
5. Atualize a variável global somando o novo valor, através do operador de atribuição composta (`+=`).
6. Escreva o valor de `totalCarrinho` atualizado no elemento visual da interface.

Exercício 10

A robustez na manipulação de eventos atinge o seu pico quando aliada à validação lógica rigorosa. A avaliação em tempo real é operada pelo evento `"input"`. Para avaliar padrões complexos de caracteres (como garantir a existência de números ou maiúsculas num texto), o JavaScript utiliza Expressões Regulares (Regex), conjugadas no controlo de fluxo lógico.

Enunciado: Construa um campo de palavra-passe que avalie a sua robustez de forma contínua à medida que é digitada. O ecrã deve indicar "Fraca", "Média" ou "Forte". Para ser "Forte", tem de cumprir três critérios: ≥ 8 caracteres, pelo menos um número e pelo menos uma letra maiúscula.

1. No HTML, insira um `<input type="password">` e um elemento de texto para providenciar *feedback* visual da robustez.
2. No JS, adicione um *Event Listener* reagindo ao evento `"input"` no campo da password.

3. Extraia o valor da `password` e efetue as validações (verifique o `.length` para o tamanho, e utilize o método `.test()` do RegEx para números `/[0-9]/` e maiúsculas `/[A-Z]/`).
4. Estructure a lógica com `if / else if`: se passar nos três testes lógicos (utilizando o operador `&&`), exiba "Forte"; se cumprir parcialmente, classifique como "Média"; caso contrário, "Frac".
5. Modifique a propriedade `style.color` para reforçar o aviso (verde, laranja ou vermelho).